Полугруппа
Моноид
Функтор

Полугруппа

Полугруппа

1) Полугруппа - множество с заданной на нём ассоциативной бинарной операцией

Полугруппа

1) Полугруппа - множество с заданной на нём ассоциативной бинарной операцией

2) Ассоциативная бинарная операция - $a \circ (b \circ c) = (a \circ b) \circ c$

Полугруппа

1) Полугруппа - множество с заданной на нём ассоциативной бинарной операцией

2) Ассоциативная бинарная операция - $a \circ (b \circ c) = (a \circ b) \circ c$

3) Замкнутость - результат применения операции к двум элементам множества принадлежит этому же множеству

Полугруппа

$$a \circ (b \circ c) = (a \circ b) \circ c$$

Примеры полугрупп:

1) Минимум в множестве целых чисел
2) Множество строк с операцией конкатенации
3) Множество натуральных чисел с операцией сложения
4) Множество натуральных чисел с операцией умножения

Полугруппа

$$a \circ (b \circ c) = (a \circ b) \circ c$$

```
min (min 3 9) 4 == min 3 (min 9 4)   -- 3 == 3
```

Semigroup

```
ghci> :i Semigroup
type Semigroup :: * → Constraint
class Semigroup a where
  (<>) :: a → a → a
  -- …
  {-# MINIMAL (<>) | sconcat #-}
```

Semigroup. Min

```haskell
-- или уже готовый
-- import Data.Semigroup (Min)

newtype Min = Min Int
  deriving (Eq, Show)

instance Semigroup Min where
  (<>) :: Min → Min → Min
  Min x <> Min y
    | x ⩽ y    = Min x
    | otherwise = Min y
```

```
ghci> Min 5 <> Min 2 <> Min 53 <> Min 3
Min 2
```

Semigroup. Sum

```haskell
-- или уже готовый
-- import Data.Semigroup (Sum)

newtype Sum = Sum Int
  deriving (Eq, Show)

instance Semigroup Sum where
  (<>) :: Sum → Sum → Sum
  Sum x <> Sum y = Sum (x + y)
```

```
ghci> Sum 5 <> Sum 2 <> Sum 53 <> Sum 3
Sum 63
```

Semigroup. (,), [], Ordering

```
ghci> (Sum 1, Min 4) <> (Sum 2, Min 1) <> (Sum 3, Min 12)
(Sum 6,Min 1)
```

```
ghci> [1, 2] <> [3, 4]
[1,2,3,4]
```

```
ghci> [Sum 1, Sum 2] <> [Sum 3, Sum 4]
[Sum 1,Sum 2,Sum 3,Sum 4]
```

```
-- передаем привет findMostProfitable
ghci> compare 1 1 <> compare 3 2 <> compare 0 5
GT
ghci> compare 2 2 <> compare 4 4 <> compare 1 8
LT
```

Моноид

1) Моноид — полугруппа с нейтральным элементом

2) Нейтральный элемент e - $e \circ x = x = x \circ e$

3) Любую полугруппу можно превратить в моноид, определив нейтральный элемент

Monoid

```
ghci> :i Monoid
type Monoid :: * → Constraint
class Semigroup a ⇒ Monoid a where
  mempty :: a
  mappend :: a → a → a
  mconcat :: [a] → a
  {-# MINIMAL mempty | mconcat #-}
```

- mappend более старая версия <>

Monoid. Min

```haskell
newtype Min = Min Int
  deriving (Eq, Show)

instance Semigroup Min where
  (<>) :: Min → Min → Min
  Min x <> Min y
    | x ⩽ y    = Min x
    | otherwise = Min y

instance Monoid Min where
  mempty :: Min
  mempty = Min maxBound
```

```
ghci> mempty <> Min 5 == Min 5 <> mempty
True
ghci> mempty <> Min 5 == Min 5
True
```

Monoid. Sum

```haskell
newtype Sum = Sum Int
  deriving (Eq, Show)

instance Semigroup Sum where
  (<>) :: Sum → Sum → Sum
  Sum x <> Sum y = Sum (x + y)

instance Monoid Sum where
  mempty :: Sum
  mempty = Sum 0
```

```
ghci> mempty <> Sum 5 == Sum 5
True
ghci> mempty <> Sum 5 == Sum 5 <> mempty
True
```

Monoid. Применения нейтрального элемента

```
ghci> foldr (+) 0 [1, 2, 3, 4]
10
```

Monoid. Применения нейтрального элемента

```
ghci> foldr (+) 0 [1, 2, 3, 4]
10
```

```
ghci> foldr (<>) mempty [Sum 1, Sum 2, Sum 3, Sum 4]
Sum 10
```

Monoid. Применения нейтрального элемента

```
ghci> foldr (+) 0 [1, 2, 3, 4]
10
```

```
ghci> foldr (<>) mempty [Sum 1, Sum 2, Sum 3, Sum 4]
Sum 10
```

```
ghci> :t mconcat
mconcat :: Monoid a ⇒ [a] → a
ghci> mconcat [Sum 1, Sum 2, Sum 3, Sum 4]
Sum 10
```

Monoid. →

```
instance Semigroup b ⇒ Semigroup (a → b)

instance Monoid b ⇒ Monoid (a → b)
```

Monoid. →

```
instance Semigroup b ⇒ Semigroup (a → b)

instance Monoid b ⇒ Monoid (a → b)
```

1) Каждый элемент множества - функция
2) Каждая функция принимает значение из множества `a`
3) Каждая функция возвращает элемент множества `b`. При этом множество `b` является полугруппой / моноидом

Monoid. →

```
instance Semigroup b ⇒ Semigroup (a → b) where
  f <> g =
```

Как соблюсти свойство полугруппы и
корректно реализовать оператор <> ?

Monoid. →

```
instance Semigroup b ⇒ Semigroup (a → b) where
  f <> g = \x → f x <> g x
```

Monoid. →

```
instance Semigroup b ⇒ Semigroup (a → b) where
  f <> g = \x → f x <> g x
```

А что будет нейтральным элементом?

```
instance Monoid b ⇒ Monoid (a → b) where
  mempty =
```

Monoid. →

```
instance Semigroup b ⇒ Semigroup (a → b) where
  f <> g = \x → f x <> g x
```

```
instance Monoid b ⇒ Monoid (a → b) where
  mempty = \_ → mempty
```

Monoid. →. Проверка свойств полугруппы

```haskell
import Data.Monoid (All(..))

positive, evenInt, small :: Int → All

positive x = All (x > 0)
evenInt  x = All (even x)
small    x = All (x < 100)
```

```
ghci> positive <> (evenInt <> small) $ 5
All {getAll = False}
ghci> (positive <> evenInt) <> small $ 5
All {getAll = False}
ghci> positive <> evenInt <> small $ 5
All {getAll = False}
```

Monoid. →. Проверка свойств моноида

```haskell
import Data.Monoid (All(..))

positive, evenInt, small :: Int → All

positive x = All (x > 0)
evenInt  x = All (even x)
small    x = All (x < 100)
```

```
ghci> positive <> mempty $ 5
All {getAll = True}
ghci> mempty <> positive $ 5
All {getAll = True}
ghci> positive $ 5
All {getAll = True}
```

Monoid. →

1) Функция вида a → b - морфизм из a в b
   - Если результат функции моноид, то моноид на таком множестве - поточечный моноид (стандартная реализация)

2) Функция вида a → a - эндоморфизм
   - Моноид над таким множеством - моноид эндоморфизмов
   - см. Data.Monoid.Endo

Monoid. →

1) Функция вида a → b - морфизм из a в b
   - Если результат функции моноид, то моноид на таком множестве - поточечный моноид (стандартная реализация)

2) Функция вида a → a - эндоморфизм
   - Моноид над таким множеством - моноид эндоморфизмов
   - см. Data.Monoid.Endo

Ничего не понятно, но крайне интересно?

https://github.com/OrelSokolov/milewski-ctfp-pdf-russian

Functor

```
class Functor f where
  fmap :: (a → b) → f a → f b
```

Functor

```
class Functor f where
  fmap :: (a → b) → f a → f b
```

Что такое f?

*, :k

Есть 3 уровня

1. значения (Term/Value) `5::Int`
2. типы (Type) `Int, String`
3. род (тип типа) (Kind) `Int :: *, String :: *`

```
ghci> :k Int
Int :: *
ghci> :k []
[] :: * → *
ghci> :k (,,)
(,,) :: * → * → * → *
ghci> :k [Int]
[Int] :: *
ghci> :k (Int, Char, Integer)
(Int, Char, Integer) :: *
```

Functor

```
class Functor f where
  fmap :: (a → b) → f a → f b
```

Что такое `f`?

`f :: * → *`

Functor

```haskell
class Functor f where
  fmap :: (a → b) → f a → f b
```

```haskell
class Functor f where
  fmap :: (a → b) → (f a → f b)
```

Functor

```
class Functor f where
  fmap :: (a → b) → f a → f b
```

```
class Functor f where
  fmap :: (a → b) → (f a → f b)
```

Функтор позволяет поднять вычисление в контейнер.

- Вычисление вида `a → b` превращается в вычисление вида `f a → f b`, где f - произвольный контейнер
- fmap - как протащить наше вычисление через контейнер

Functor []

```haskell
class Functor f where
  fmap :: (a → b) → f a → f b
```

```haskell
instance Functor [] where
   fmap = map
```

```
ghci> fmap (+1) [1, 2, 3]
[2,3,4]
ghci> (+1) <$> [1, 2, 3]
[2,3,4]
ghci> (+1) <$> (^2) <$> [1, 2, 3]
[2,5,10]
```

Functor Maybe

```haskell
class Functor f where
  fmap :: (a → b) → f a → f b
```

```haskell
instance Functor Maybe where
  fmap _ Nothing = Nothing
  fmap f (Just x) = Just (f x)
```

```
ghci> fmap (+1) Nothing
Nothing
ghci> fmap (+1) $ Just 5
Just 6
```

Functor Tree

```haskell
class Functor f where
  fmap :: (a → b) → f a → f b
```

```haskell
data Tree a = Leaf a | Branch (Tree a) a (Tree a)
  deriving Show

instance Functor Tree where
  fmap f (Leaf x) = Leaf (f x)
  fmap f (Branch left x right) = Branch (fmap f left) (f x) (fmap f
right)
```

```
ghci> let tree = Branch (Leaf 1) 2 (Leaf 3)
ghci> fmap even tree
Branch (Leaf False) True (Leaf False)
```

Functor. (,)

```
-- Expected kind ‘* → *’, but ‘(,)’ has kind ‘* → * → *’
instance Functor ((,)) where
  fmap = …
```

Functor. (,)

```
-- Expected kind ‘* → *’, but ‘(,)’ has kind ‘* → * → *’
instance Functor ((,)) where
  fmap = …
```

```
-- связываем один параметр
instance Functor ((,) s) where
--fmap :: (a → b) → (f a → f b)
  fmap :: (a → b) → (s, a) → (s, b)
  fmap f (x, y) = (x, f y)
```

```
ghci> fmap (+1) (1, 2)
(1,3)
```

Functor. →

```haskell
instance Functor ((→) e) where
--fmap :: (a → b) → (f a → f b)
  fmap :: (a → b) → (e → a) → (e → b)
  fmap = (.)
```

```
ghci> :t fmap (+1) (*2)
fmap (+1) (*2) :: Num b ⇒ b → b
ghci> fmap (+1) (*2) 10
21
```

Functor. Законы

1. `fmap id == id`
   - `fmap id [1,2,3] == id [1,2,3]`

2. `fmap f . fmap g == fmap (f . g)`
   - `(fmap (+1) . fmap (*2)) [1,2,3] == fmap ((+1) . (*2)) [1,2,3]`



ВСЁ!